Computer Architecture:
The Stack Frame – ISA Meeting The Compiler

Dr. Colin O’Flynn

Lectures

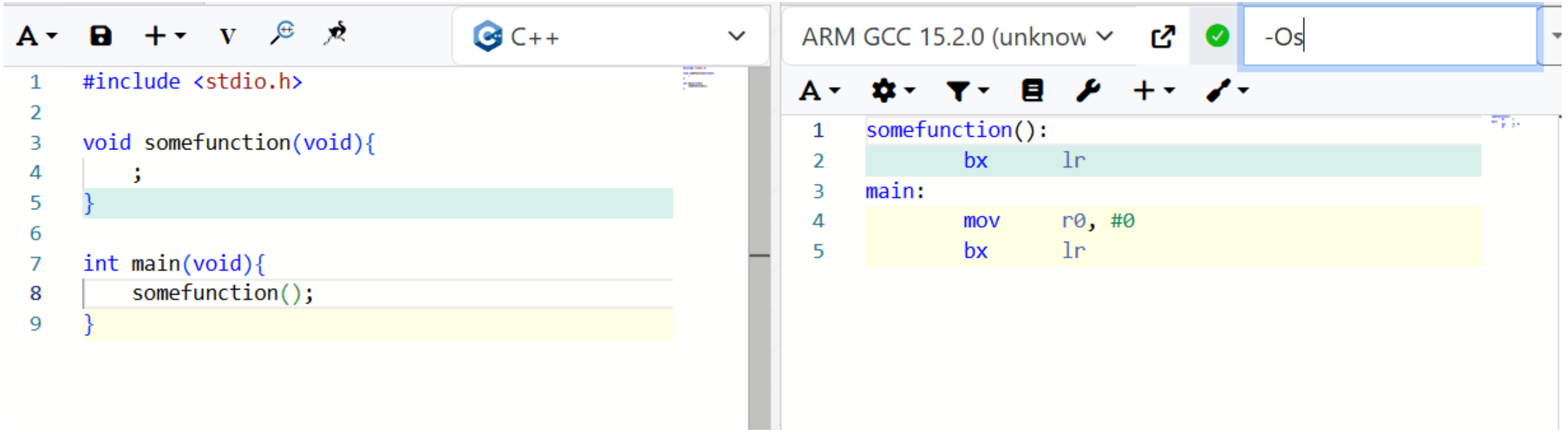
- How does a program run on a machine?
- How do you compile from high-level code to machine code?
- How do you observe the machine running?
- How do we define the machine?
- **How does the machine control flow change (planned)?**
- How does the machine control flow change (unplanned)?

...at some point: what is the *most basic possible* machine...

Link to Debuggers

- Now that you have a working debugger (or will have one shortly!), you can observe the *exact details of the machine operations*.

Calling a Function



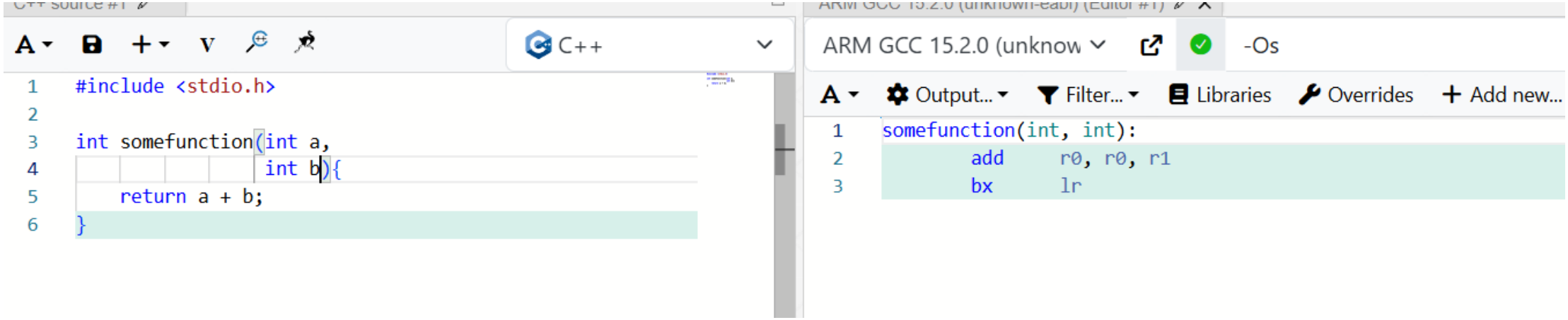
Arm Calling Notes

- LR (Link Register) = Return Address

- bl = Branch with Link (copy address to LR)

NOTE: The “lr” is not something that exists on all platforms – x86/x86-64 *does not* have this idea and stores the return address on the stack (more on that…)

Calling Conventions

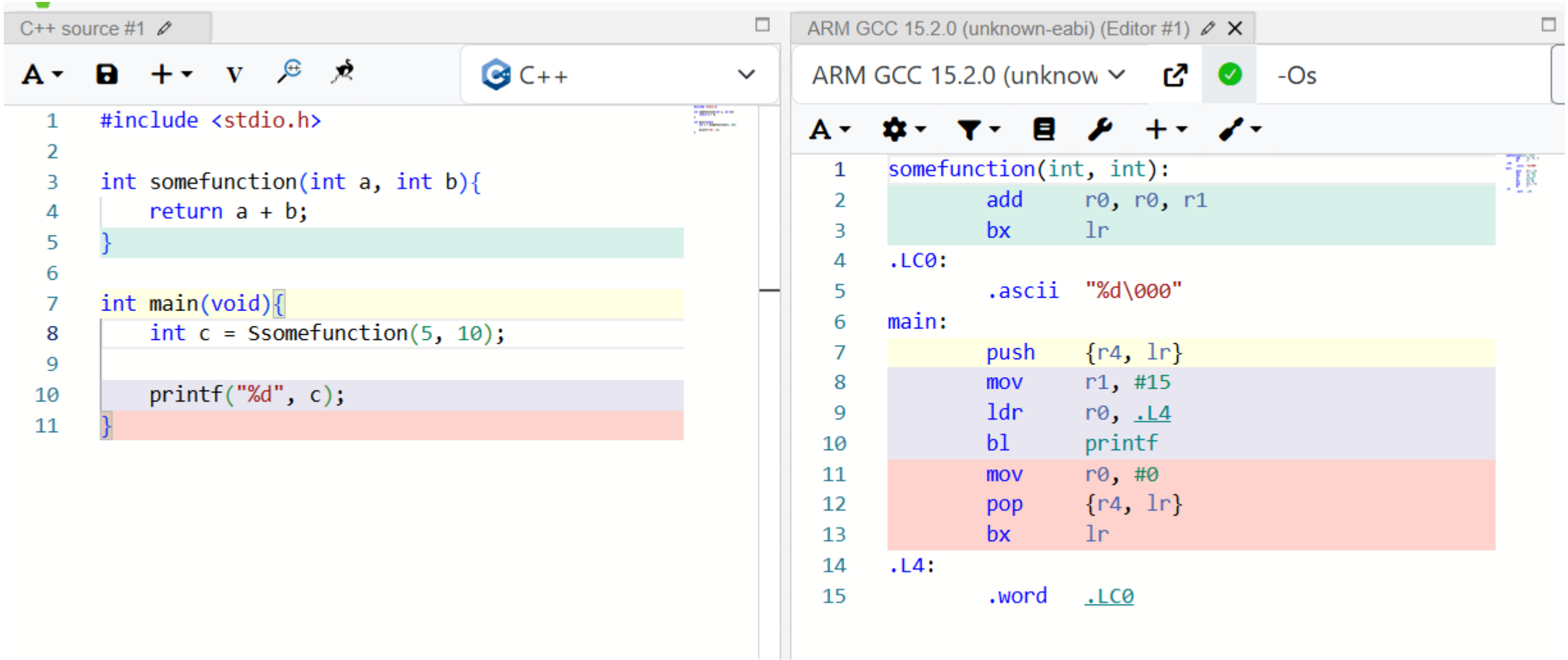


Guess so far

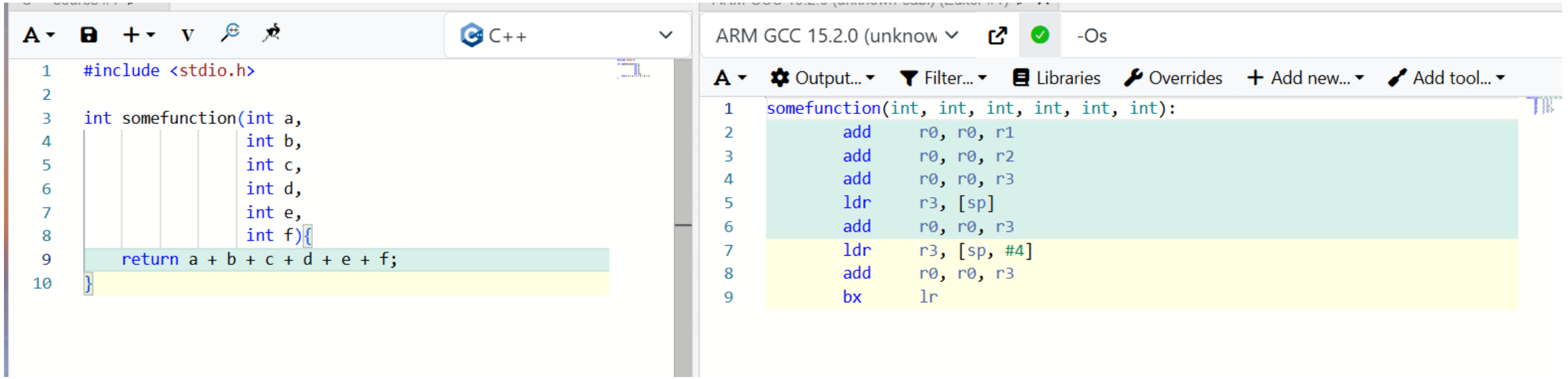
- Passed arguments:
  - R0 = Argument 0
  - R1 = Argument 1

- Return values:
  - R0 = Return value

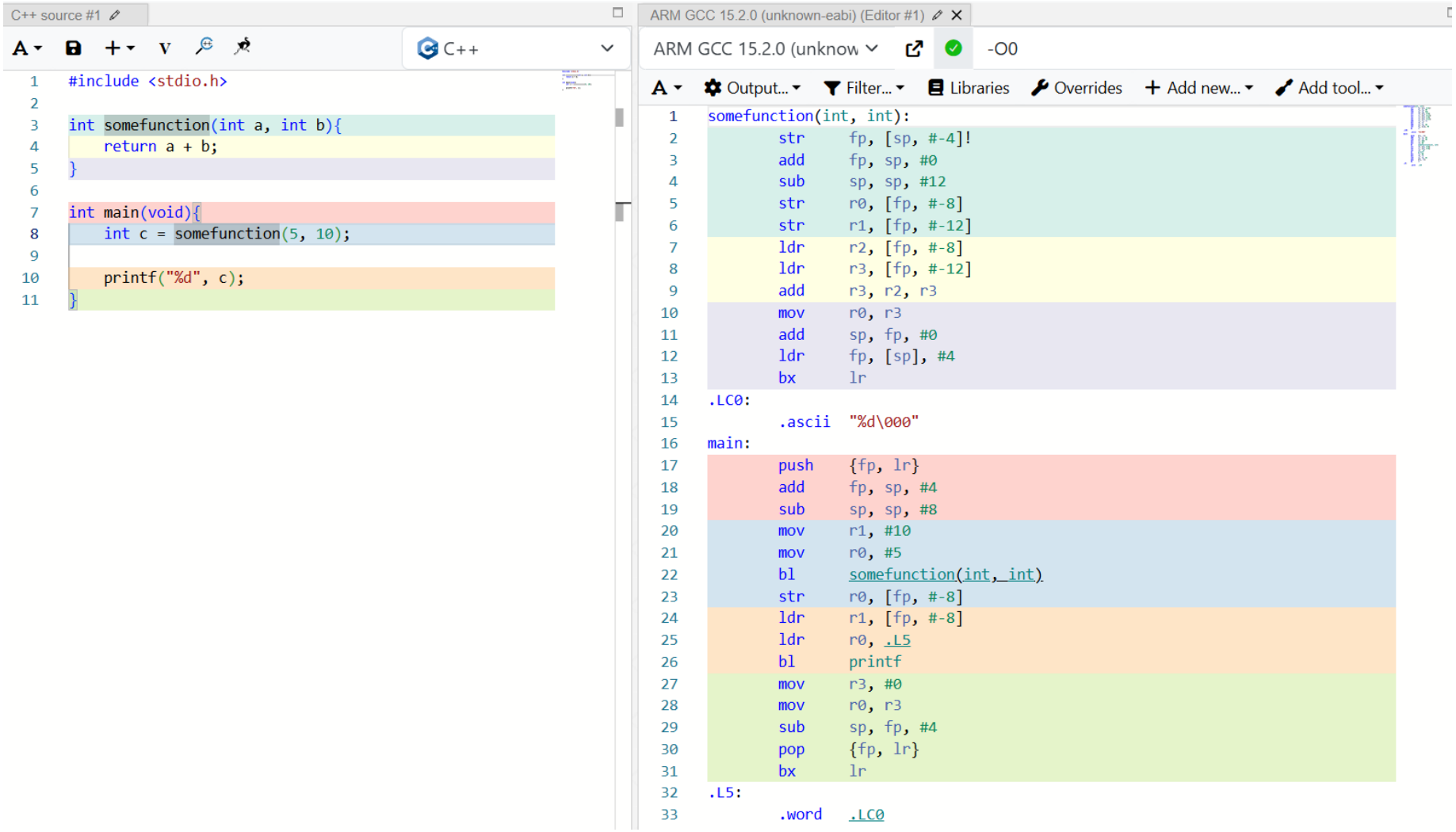
Sidenote: calling conventions & -Os



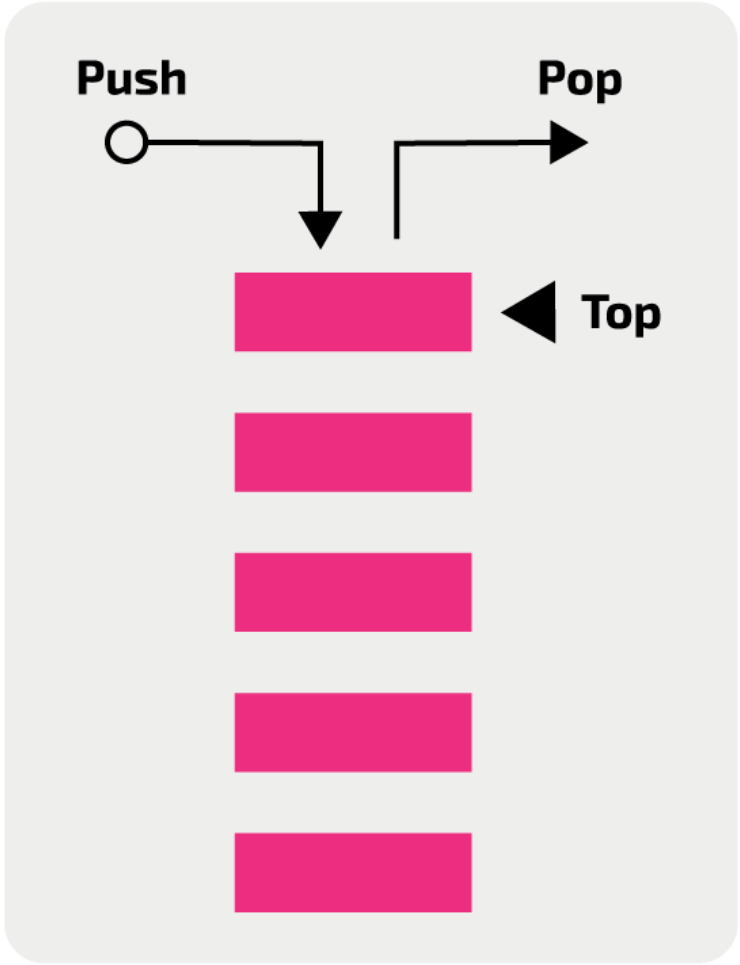
...what is “sp”


C++
1 #include <stdio.h>
2
3 int somefunction(int a,
4 int b,
5 int c,
6 int d,
7 int e,
8 int f){
9 return a + b + c + d + e + f;
10 }
ARM GCC 15.2.0 (unknow
-Os
Output...
Filter...
Libraries
Overrides
Add new...
Add tool...
1 somefunction(int, int, int, int, int, int):
2 add r0, r0, r1
3 add r0, r0, r2
4 add r0, r0, r3
5 ldr r3, [sp]
6 add r0, r0, r3
7 ldr r3, [sp, #4]
8 add r0, r0, r3
9 bx lr

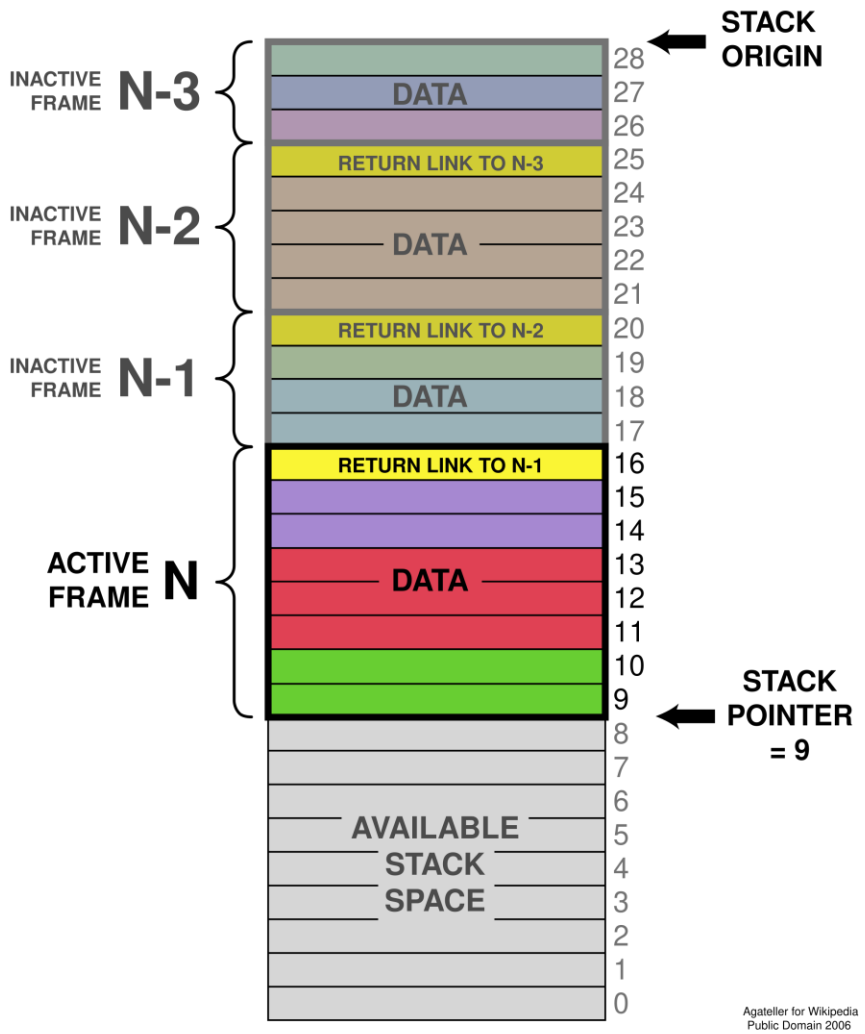
The Stack


C++ source #1
C++
1 #include <stdio.h>
2
3 int somefunction(int a, int b){
4 return a + b;
5 }
6
7 int main(void){
8 int c = somefunction(5, 10);
9
10 printf("%d", c);
11 }
ARM GCC 15.2.0 (unknown-eabi) (Editor #1)
ARM GCC 15.2.0 (unknow
-O0
Output...
Filter...
Libraries
Overrides
Add new...
Add tool...
1 somefunction(int, int):
2 str fp, [sp, #-4]!
3 add fp, sp, #0
4 sub sp, sp, #12
5 str r0, [fp, #-8]
6 str r1, [fp, #-12]
7 ldr r2, [fp, #-8]
8 ldr r3, [fp, #-12]
9 add r3, r2, r3
10 mov r0, r3
11 add sp, fp, #0
12 ldr fp, [sp], #4
13 bx lr
14 .LC0:
15 .ascii "%d\000"
16 main:
17 push {fp, lr}
18 add fp, sp, #4
19 sub sp, sp, #8
20 mov r1, #10
21 mov r0, #5
22 bl somefunction(int, int)
23 str r0, [fp, #-8]
24 ldr r1, [fp, #-8]
25 ldr r0, .L5
26 bl printf
27 mov r3, #0
28 mov r0, r3
29 sub sp, fp, #4
30 pop {fp, lr}
31 bx lr
32 .L5:
33 .word .LC0

The Stack



The Stack


INACTIVE FRAME N-3
INACTIVE FRAME N-2
INACTIVE FRAME N-1
ACTIVE FRAME N
DATA
RETURN LINK TO N-3
DATA
RETURN LINK TO N-2
DATA
RETURN LINK TO N-1
DATA
AVAILABLE STACK SPACE
28
27
26
25
24
23
22
21
20
19
18
17
16
15
14
13
12
11
10
9
8
7
6
5
4
3
2
1
0
STACK ORIGIN
STACK POINTER = 9
Agateller for Wikipedia
Public Domain 2006

The Stack

IMPORTANT: Stack is just memory (RAM). We give it a special name but nothing stops you from read/writing to the stack.

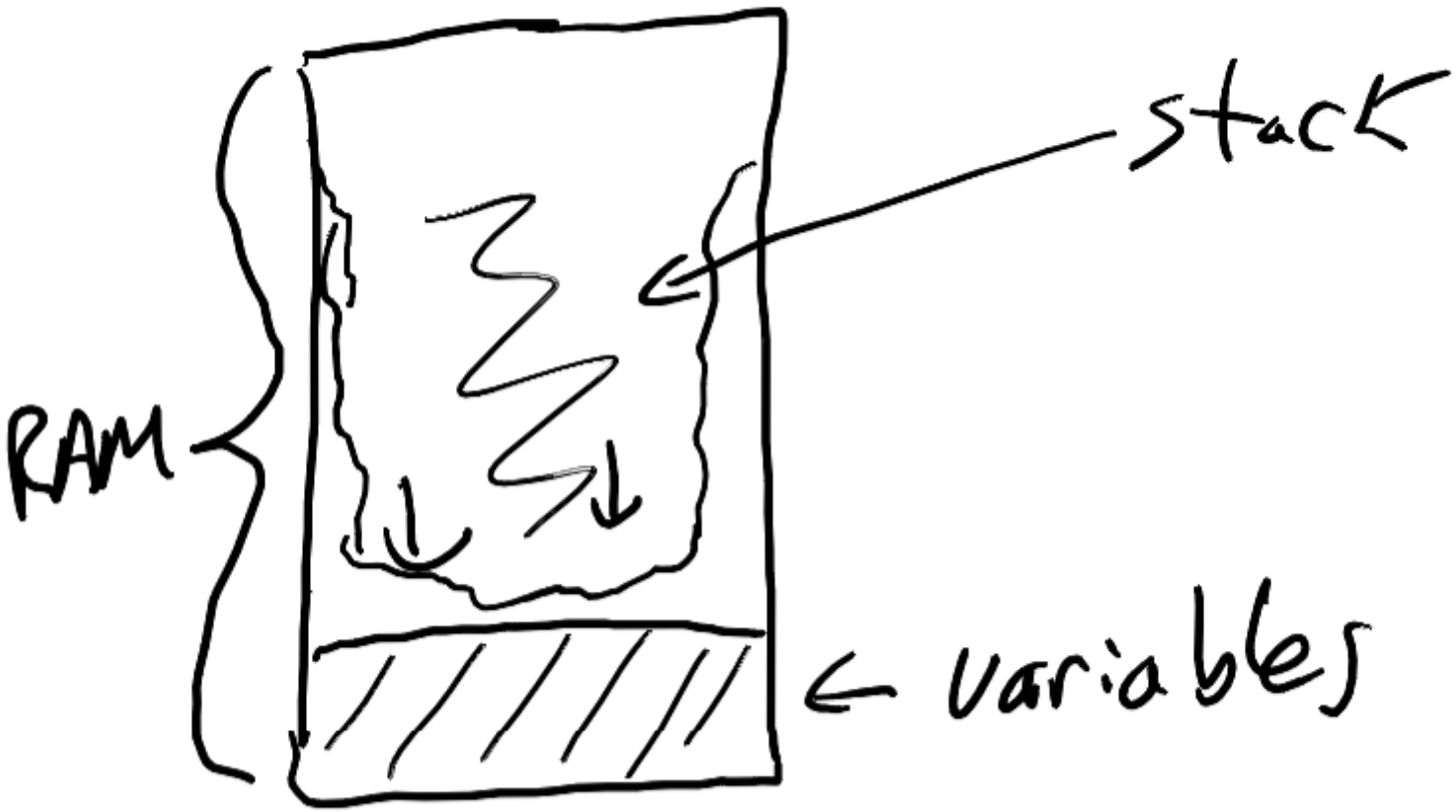
...also nothing stops the stack space from read/writing into memory...

Stack Growth

- Stack normally grows downward on newer embedded platforms (Arm/AVR), but check on others

Stack & Variable Storage?

- Stack also stores local variables.
- **WARNING**: Stack can collide with memory – this results in memory corruption and other issues.
- **CHECKING STACK SIZE**: You can write known patterns to memory & inspect how far “overwritten” they got:


stack
RAM
variables

Prologue & Epilogue

C++ source #1

```c
#include <stdio.h>

int somefunction(int a, int b){
    return a + b;
}

int main(void){
    int c = somefunction(5, 10);

    printf("%d", c);
}
```

ARM GCC 15.2.0 (unknown-eabi) (Editor #1)

ARM GCC 15.2.0 (unknow -O0

Output... Filter... Libraries Overrides Add new... Add tool...

```
somefunction(int, int):
        str     fp, [sp, #-4]!
        add     fp, sp, #0
        sub     sp, sp, #12
        str     r0, [fp, #-8]
        str     r1, [fp, #-12]
        ldr     r2, [fp, #-8]
        ldr     r3, [fp, #-12]
        add     r3, r2, r3
        mov     r0, r3
        add     sp, fp, #0
        ldr     fp, [sp], #4
        bx      lr
.LC0:
        .ascii  "%d\000"
main:
        push    {fp, lr}
        add     fp, sp, #4
        sub     sp, sp, #8
        mov     r1, #10
        mov     r0, #5
        bl      somefunction(int, int)
        str     r0, [fp, #-8]
        ldr     r1, [fp, #-8]
        ldr     r0, .L5
        bl      printf
        mov     r3, #0
        mov     r0, r3
        sub     sp, fp, #4
        pop     {fp, lr}
        bx      lr
.L5:
        .word   .LC0
```

Prologue

Epilogue

Copy/Paste Slide (Ignore)

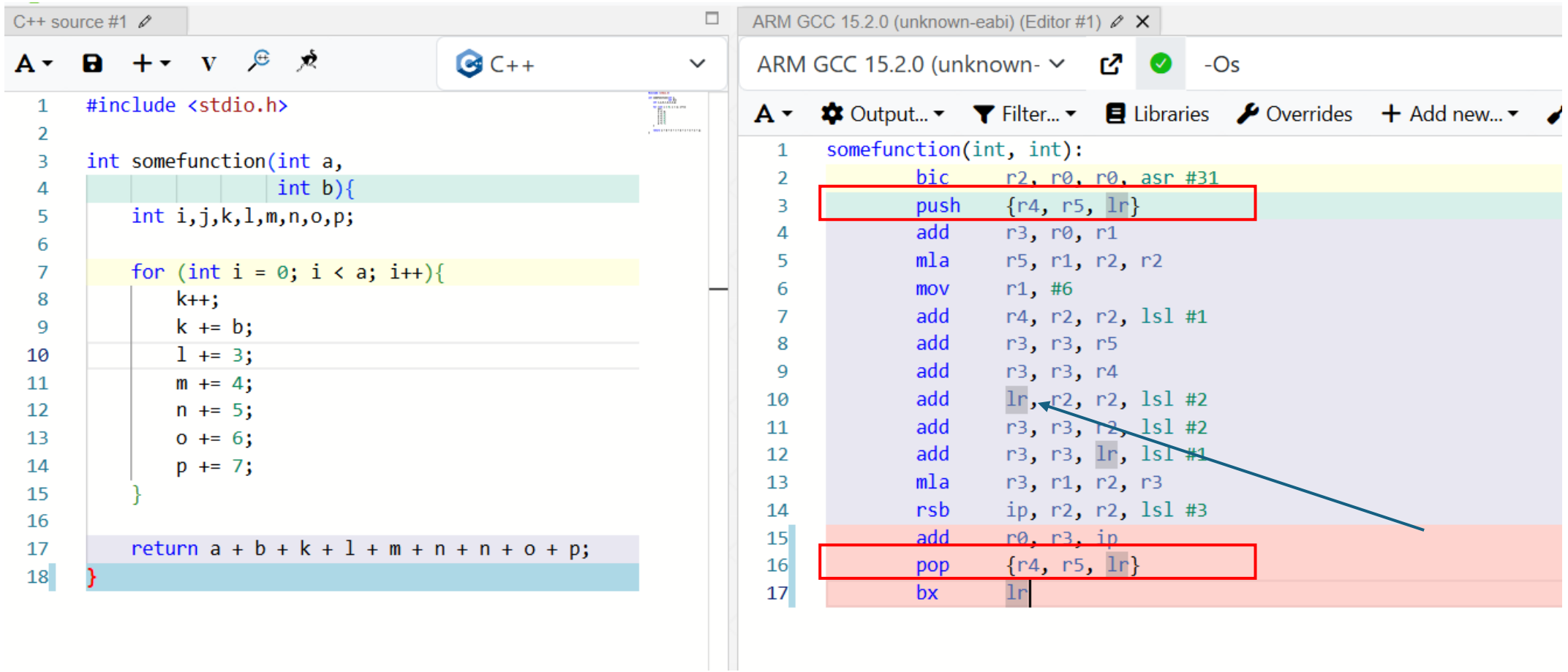
```
#include <stdio.h>


int somefunction(int a,
                 int b){
    int i,j,k,l,m,n,o,p;


    for (int i = 0; i < a; i++){
        k++;
        k += b;
        l += 3;
        m += 4;
        n += 5;
        o += 6;
        p += 7;
    }


    return a + b + k + l + m + n + n + o + p;
}
```

Caller-saved Registers



Callee-saved Registers

```c
#include <stdio.h>

int somefunction(int a,
                 int b){
    int i,j,k,l,m,n,o,p;

    a = somefunction(a, b);

    for (int i = 0; i < a; i++){
        k++;
        k += b;
        l += 3;
        m += 4;
        n += 5;
        o += 6;
        p += 7;
    }



    return a + b + k + l + m + n + n + o + p;
}
```

Output... Filter... Libraries Overrides Add new...

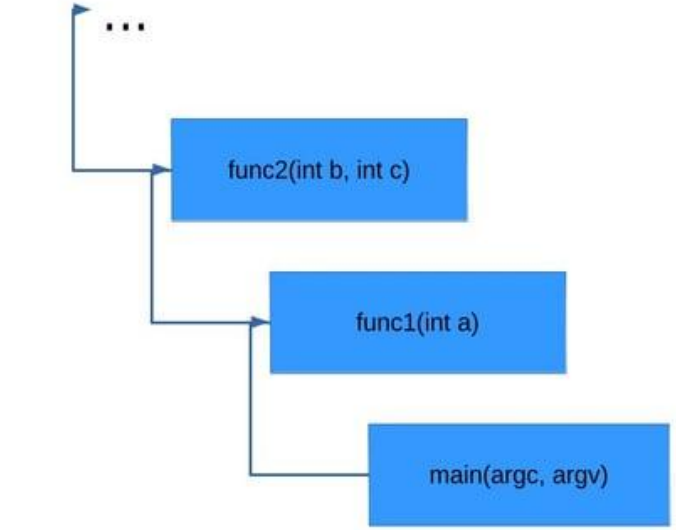
```
somefunction(int, int):
        push    {r4, r5, r6, lr}
        mov     r4, r1
        bl      somefunction(int, int)
        bic     r2, r0, r0, asr #31
        add     r3, r4, r0
        mov     r0, #6
        mla     r5, r4, r2, r2
        add     lr, r2, r2, lsl #1
        add     r3, r3, r5
        add     r3, r3, lr
        add     ip, r2, r2, lsl #2
        add     r3, r3, r2, lsl #2
        add     r3, r3, ip, lsl #1
        mla     r0, r2, r0, r3
        rsb     r1, r2, r2, lsl #3
        add     r0, r0, r1
        pop     {r4, r5, r6, lr}
        bx      lr
```

ARM Cortex-M Calling Conventions & Registers

- **Arguments:**
  - **First 4**: r0 – r3
  - **Remaining**: stack
- **Return value** in r0
- **Registers**:
  - Caller-saved: r0-r3, r12
    - *HINT: r0-r3… see above*
  - Callee-saved r4-r11
- **Link Register (LR)**
  - **R14**
  - May be reused as a “regular register”
- **Intra-procedure-call scratch register (IP)**
  - **R12**
  - May be reused as a “regular register”

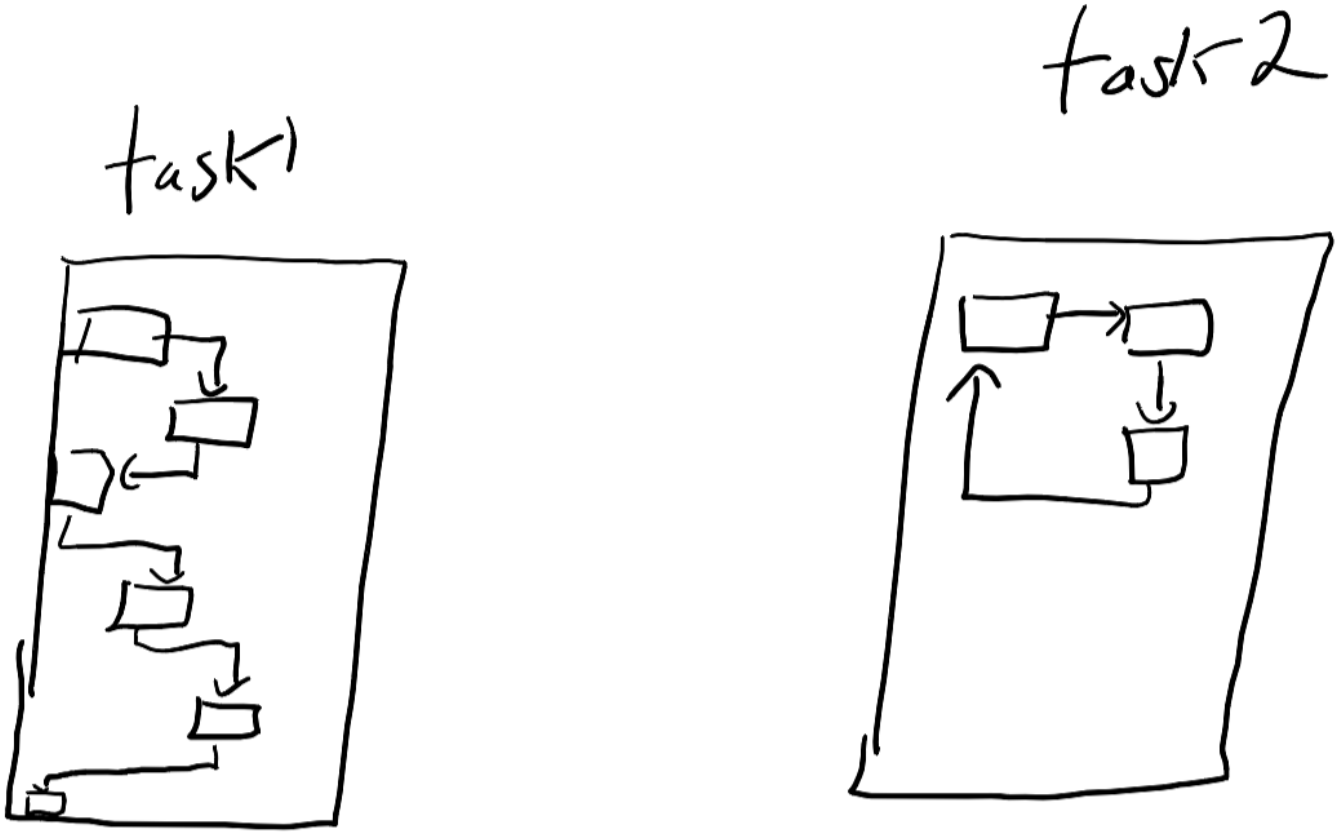
Stack Trace & Debuggers

```
#0  func3 () at main.c:2
#1  0x0000555555555150 in func2 (c=7, b=8) at main.c:7
#2  0x0000555555555171 in func1 (a=5) at main.c:11
#3  0x0000555555555191 in main (argc=1, argv=0x7fffffffe138) at main.c:15
```



Remember: each of these is a *Stack Frame*

Stack Frames & Context Switching



Context Switching

- We can *save* and *restore* the system context:
  - All registers
  - Naturally saves & restores stack pointer
  - Requires us to setup memory segments so these do not overlap

Lab & Assignment Explorations

- Lab #1 is about debugger usage – but you can also take a look at stack frames in the debugger.

- You'll be asked to look in more detail of these stack frames in later Labs & assignments, but I suggest taking a look at it in your first lab to get an understanding for the tools.

Stack Summary

- The *Stack* is memory used by functions:
  - Arguments
  - Local storage
  - Return address (sometimes)
- On most systems nothing stops *stack overrun*
- The *Stack Frame* is a specific section of memory used by each function.
  - We can trace through Stack Frames to understand call order (“stack trace”)
- Callee vs. Caller-Saved
  - Critical to understand who is responsible for saving register state
  - Caller-saved registers normally used for argument passing anyway